Probeklausur: Mikrocomputertechnik 1

Bearbeitungszeit: 60 Minuten · Gesamtpunktzahl: 60 Punkte

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1 enthält zehn Fragen im Multiple-Choice-Format. Bei jeder Frage können eine, mehrere oder alle Antwortmöglichkeiten richtig sein. Jede vollständig korrekte Auswahl erhält einen Punkt. Die Aufgaben 2–7 sind Papieraufgaben mit den jeweils angegebenen Punkten; dort sind Teilpunkte möglich.

Musterlösung mit Bewertungshinweisen

Name:

_____

Matrikelnummer:

_____

Kurs / Studiengruppe:

_____

Hilfsmittel: Das offizielle RISC-V Green Card / Reference Sheet wird ausgeteilt. Alle Aufgaben sind auf Papier ohne Simulator oder Compiler lösbar.

Stoffumfang: Vorlesungen 1–7 und deren Wiederholung in Einheit 9.

Markieren Sie in Aufgabe 1 alle richtigen und keine falschen Optionen. Jede andere Auswahl erhält null Punkte; es gibt keinen Punktabzug. Geben Sie bei den Aufgaben 2–7 Rechenwege beziehungsweise Begründungen an.

Aufgabe	Inhalt	Punkte	Richtzeit	erreicht
1	10 Multiple-Choice-Fragen, Vorlesungen 1–7	10	10 min	
2	Zahlendarstellung und Logik	8	8 min	
3	Array und Schleife in RV32I	10	10 min	
4	Funktionsaufruf und Stacktrace	8	8 min	
5	Datenpfad und Steuerwerk	8	8 min	
6	Pipeline und Laufzeit	10	10 min	
7	Maschinencode und Bytefolge	6	6 min	
	Summe	**60**	**60 min**	

Die Richtzeiten umfassen Lesen und Bearbeiten und dienen der Zeiteinteilung.

Gemeinsame Annahmen: RV32I ohne komprimierte Instruktionen, 32-Bit-Register, Little Endian und Byteadressierung. Echte Instruktionen sind vier Byte lang. Registerarithmetik behält die unteren 32 Bit. Normale Funktionsaufrufe verwenden die ILP32-Konvention: skalare Ergebnisse in `a0`, Stackzeiger während standardkonformer Funktionen auf 16 Byte ausgerichtet. Die Arrayelemente sind 32-Bit-Wörter. Datenadressen und Speicherbereiche sind gültig, soweit nicht ausdrücklich nach einer Bereichsgrenze gefragt wird.

Datenpfad und Pipeline sind die jeweils ausdrücklich angegebenen Lehrmodelle. Zeit- und Adresswerte sind Aufgabenannahmen, keine gemessenen Eigenschaften eines Prozessors oder Simulators.

Aufgabe 1: Multiple Choice (10 Punkte)

Fragen 1–4

Frage 1: ISA und Umsetzung (1 Punkt)

Zwei Prozessoren führen denselben RV32I-Code korrekt aus, benötigen dafür aber verschieden viele Takte. Welche Aussagen treffen zu?

- ☐ A: Beide können dieselbe ISA mit unterschiedlichen Mikroarchitekturen implementieren.
- ☐ B: Die unterschiedliche Taktzahl verlangt unterschiedliche RV32I-Instruktionscodierungen.
- ☐ C: Die Laufzeit lässt sich allein aus den Assemblerzeilen bestimmen.
- ☐ D: Die ISA beschreibt das sichtbare Verhalten, nicht jede interne Taktfolge.

Frage 2: Bitmuster und Zahlenwert (1 Punkt)

Das achtbittige Muster `1111 1011` wird ohne Änderung der Bits betrachtet. Welche Aussagen treffen zu?

- ☐ A: Als vorzeichenloser Wert stellt es die Zahl 251 dar.
- ☐ B: Als Zweierkomplementwert stellt es die Zahl −5 dar.
- ☐ C: Mit Vorzeichenerweiterung wird daraus `0xFFFFFFFB`.
- ☐ D: Mit Nullerweiterung wird daraus `0x000000FB`.

Frage 3: Logik und gespeicherter Zustand (1 Punkt)

Ein Multiplexer und ein positiv flankengetriggertes Register werden ohne Reset betrachtet. Welche Aussagen treffen zu?

- ☐ A: Ein Multiplexer kann seinen Ausgang nach einer Auswahländerung zwischen Flanken ändern.
- ☐ B: Ein Multiplexer übernimmt seine neue Auswahl erst an der nächsten Taktflanke.
- ☐ C: Das Register folgt bei dauerhaft hohem Takt jeder Änderung seines Dateneingangs.
- ☐ D: Ohne neue aktive Flanke bleibt der gespeicherte Registerwert erhalten.

Frage 4: Register und Immediates (1 Punkt)

Für RV32I ohne Erweiterungen: Welche Aussagen treffen zu?

- ☐ A: Nach `addi x0, x0, 7` enthält das Register `x0` den Wert sieben.
- ☐ B: Das Immediate −2048 ist bei `addi` direkt codierbar.
- ☐ C: Die Anweisung `addi t0, t1, 2048` verwendet ein zulässiges Immediate.
- ☐ D: Eine Addition über die 32-Bit-Grenze hinaus löst einen Überlauf-Trap aus.

Fragen 5–8

Frage 5: Speicher und Erweiterung (1 Punkt)

Das Wort `0x00000080` liegt ab Adresse `0x2000` in Little Endian. Die Zugriffe sind gültig. Welche Aussagen treffen zu?

- ☐ A: An Adresse `0x2000` liegt das Byte `0x00`.
- ☐ B: `lb t0, 0(a0)` mit `a0=0x2000` liefert `0xFFFFFF80`.
- ☐ C: `lbu t0, 0(a0)` mit `a0=0x2000` liefert `0x00000080`.
- ☐ D: Das Basisregister `a0` wird durch den Load um ein Byte erhöht.

Frage 6: Bedingte Sprünge (1 Punkt)

Vor dem Branch gilt `t0=0xFFFFFFFF` und `t1=1`. Welche Aussagen treffen zu?

- ☐ A: `blt t0, t1, ziel` nimmt den Branch.
- ☐ B: `bltu t0, t1, ziel` nimmt den Branch.
- ☐ C: `bgeu t0, t1, ziel` nimmt den Branch.
- ☐ D: `beq t0, t1, ziel` nimmt den Branch.

Frage 7: Aufrufkonvention (1 Punkt)

Eine normal aufgerufene ILP32-Funktion verändert `s0` und ruft mit `jal ra, helfer` eine weitere Funktion auf. Welche Aussagen treffen zu?

- ☐ A: Der Helfer muss alle vom Aufrufer noch benötigten `t`-Register erhalten.
- ☐ B: Sie muss ihren eingehenden `s0`-Wert vor der eigenen Rückkehr wiederherstellen.
- ☐ C: Sie muss ihre äußere Rückkehradresse über den Helferaufruf hinweg erhalten.
- ☐ D: Das Pseudokommando `ret` stellt den alten Stackpointer automatisch wieder her.

Frage 8: Hauptsteuerung und ALU-Steuerung (1 Punkt)

Im Kursmodell werden `add` und `sub` mit gleichem Opcode, aber unterschiedlichen gültigen Funktionsfeldern ausgeführt. Welche Aussagen treffen zu?

- ☐ A: Die beiden Befehle benötigen unterschiedliche Werte des Signals `RegWrite`.
- ☐ B: Der siebenbittige Opcode unterscheidet bereits beide Rechenoperationen.
- ☐ C: Die vier Bits von `ALUControl` sind ein vorgeschriebenes RISC-V-Instruktionsfeld.
- ☐ D: Die ALU-Detailsteuerung unterscheidet sie anhand der Funktionsfelder.

Fragen 9–10

Frage 9: Laufzeitvergleich (1 Punkt)

Zwei Ausführungen haben jeweils `N=100` Maschineninstruktionen. A hat `CPI=1` und eine Taktperiode von `4 ns`, B hat `CPI=2` und `1 ns`. Welche Aussagen treffen zu?

- ☐ A: Die Ausführung A benötigt insgesamt eine Zeit von 400 ns.
- ☐ B: Die Ausführung A ist wegen ihres kleineren CPI schneller.
- ☐ C: Die Ausführung B benötigt insgesamt eine Zeit von 200 ns.
- ☐ D: Der Quotient Zeit A durch Zeit B beträgt zwei.

Frage 10: Forwarding (1 Punkt)

Die Fünfstufen-Kurspipeline führt mit Forwarding `add t0, t1, t2`, `addi t0, t0, 4` und `sub a0, t0, t4` aus. Anfangs gelten `t1=6`, `t2=5` und `t4=3`. Welche Aussagen treffen zu?

- ☐ A: Der Verbraucher `sub` muss das ältere Ergebnis elf verwenden.
- ☐ B: Für `sub` hat der Treffer aus EX/MEM Vorrang vor MEM/WB.
- ☐ C: Der letzte Befehl schreibt den Wert zwölf nach `a0`.
- ☐ D: `add` und `addi` erzeugen verschiedene Versionen von `t0`.

Musterlösung zu Aufgabe 1

Bewertung: Je Frage ein Punkt ausschließlich für die exakte Auswahl aller richtigen und keiner falschen Optionen. Keine Teilpunkte, kein Punktabzug. Die Zahl richtiger Antworten ist nicht vorgegeben.

1. A, D — A und D: Die Instruction Set Architecture (ISA) legt die architektonische Befehlswirkung fest. B ist falsch: Andere Taktkosten erfordern keine andere Codierung. C ist falsch: Befehlszahl, Taktkosten und Taktperiode hängen auch von Expansion und Umsetzung ab.

2. A, B, C, D — Alle vier Optionen: Das Bitmuster hat unsigned den Wert 251, signed den Wert $251 - 256 = -5$. Vorzeichenerweiterung übernimmt das gesetzte Vorzeichenbit; Nullerweiterung füllt die oberen Stellen mit null. Die Interpretation und die Erweiterungsart sind unterschiedliche Entscheidungen.

3. A, D — A und D: Der Multiplexer ist kombinatorisch und kann seine Ausgabe nach seiner Laufzeit zwischen Taktflanken ändern. Ohne neue aktive Flanke bleibt der Registerzustand erhalten. B verwechselt Quellenauswahl mit Speicherung; C beschreibt ein pegelgesteuertes statt eines flankengetriggerten Speicherelements. Setup und Hold seien eingehalten.

4. B — Nur B: Das signed 12-Bit-Immediate reicht von -2048 bis 2047. Schreibzugriffe auf x0 werden verworfen; 2048 liegt außerhalb des Bereichs. RV32I-Ganzzahladdition behält die unteren 32 Bit und erzeugt keinen Überlauf-Trap.

5. B, C — B und C: Das niedrigstwertige Byte 0x80 liegt an der kleinsten Adresse. lb erweitert sein Bit 7 als Vorzeichen, lbu erweitert mit Nullen. A vertauscht die Byteordnung; D unterstellt eine automatische Adressänderung, die diese Loads nicht ausführen.

6. A, C — A und C: Signed ist t0 gleich minus eins und damit kleiner als eins. Unsigned ist dasselbe Muster 4294967295 und damit größer oder gleich eins. B verwendet die falsche unsigned Relation; D ist falsch, weil die Registermuster verschieden sind.

7. B, C — B und C: s0 ist callee-saved und muss bei der Rückkehr den Eintrittswert besitzen. Der innere jal überschreibt ra, daher ist die äußere Rückkehradresse anderweitig zu erhalten. A ist falsch: t-Register sind caller-saved. D ist falsch: ret springt lediglich und führt keinen Epilog aus.

8. D — Nur D: Beide Befehle haben denselben Opcode und RegWrite gleich eins. Die ALU-Detailsteuerung wertet ihre gültigen Funktionsfelder aus. A verwechselt Rechenfunktion und Schreibfreigabe; B unterstellt verschiedene Opcodes. C ist falsch, weil ALUControl eine interne Kurscodierung und kein ISA-Instruktionsfeld ist.

9. A, C, D — A, C und D: Die Zeit ist $N \cdot \text{CPI} \cdot T$, also 400 ns beziehungsweise 200 ns. Der Speedup von B gegenüber A beträgt 2. B ist als Aussage falsch: Ein kleinerer CPI allein genügt ohne Vergleich der Taktperiode nicht.

10. B, C, D — B, C und D: `add` erzeugt elf, `addi` daraus 15. Die nachfolgende Subtraktion braucht den jüngeren Wert 15 aus EX/MEM und berechnet $15 - 3 = 12$. Der ältere Wert elf kann gleichzeitig in MEM/WB stehen, darf aber nicht gewinnen. A verletzt die sequenzielle Befehlswirkung.

Aufgabe 2: Zahlendarstellung und Logik (8 Punkte)

a) Stellen Sie −37 als achtbittigen Zweierkomplementwert in Binär- und Hexadezimalschreibweise dar. Zeigen Sie den Rechenweg ausgehend von +37. (3 Punkte)

b) Addieren Sie die achtbittigen Muster `0111 1111` und `0000 0001`. Geben Sie Ergebnis und hinausfallenden Übertrag an. Liegt bei signed Interpretation ein Überlauf vor? Begründen Sie. (2 Punkte)

c) Ein Einbit-Multiplexer wählt bei $S = 0$ den Eingang A, bei $S = 1$ den Eingang B. Geben Sie die Boolesche Gleichung für Y an und füllen Sie die vier Tabellenzeilen aus. (3 Punkte)

S	A	B	Y
0	1	0	
1	1	0	
1	0	1	
0	0	1	

Musterlösung und Bewertung

a) $37 = 32 + 4 + 1$: `0010 0101`. Invertieren: `1101 1010`; eins addieren: `1101 1011`, also `0xDB`. Ein Punkt für +37, ein Punkt für die korrekte Bildung des negativen Musters, ein Punkt für den Hexwert.

b) Ergebnis `1000 0000`, hinausfallender Übertrag null. Als signed Rechnung sollen zwei nichtnegative Werte $127 + 1 = 128$ entstehen; das achtbittige Ergebnis stellt jedoch −128 dar. Damit liegt signed Überlauf vor. Ein Punkt für Ergebnis und Übertrag zusammen (je 0,5), ein Punkt für Überlauf mit Begründung (je 0,5).

c) $Y = (\overline{S} \wedge A) \vee (S \wedge B)$. Ergebnisse von oben nach unten: 1, 0, 1, 0. Ein Punkt für eine äquivalente Gleichung, je 0,5 Punkte pro Tabellenwert.

Aufgabe 3: Array und Schleife in RV32I (10 Punkte)

Die Blattfunktion `sum_first` soll die ersten `n` Elemente eines Arrays addieren und die Summe zurückgeben. Für `n=0` liefert sie null, ohne ein Arrayelement zu lesen.

Es gilt $0 \leq n \leq 4$. Ab `0x3000` liegen vier 32-Bit-Zweierkomplementwerte lückenlos: −2, 5, 7, −3. Bei Eintritt enthält `a0` die Basisadresse `0x3000`, `a1` die vorzeichenlose Länge `n`. Ein Element mit Index `i` steht vier mal `i` Byte hinter der Basis; die Indizes beginnen bei null. Die Funktion benötigt keine Unteraufrufe und darf ohne Stackframe auskommen.

a) Ergänzen Sie die sechs mit _____ markierten Lücken. `t0` hält die unveränderte Arraybasis, `t1` den Index und `t2` die Summe. Die Abbruchbedingung verwendet einen unsigned Vergleich; die Rückgabe gehört in `a0`. (6 Punkte)

```
sum_first:
    addi t0, a0, 0
    addi t1, x0, 0
    addi t2, x0, 0
sum_loop:
    ____ t1, a1, sum_end
    slli t3, t1, _____
    add  t4, _____, t3
    lw   t5, _____(t4)
    add  t2, t2, _____
    addi t1, t1, 1
    jal  x0, sum_loop
sum_end:
    addi a0, _____, 0
    jalr x0, 0(ra)
```

b) Bestimmen Sie für `n=0` und `n=3` jeweils den Rückgabewert und die Zahl der Array-Loads. (2 Punkte)

c) Geben Sie die Byteadresse und den gespeicherten Zahlenwert des Elements mit Index zwei an. Begründen Sie die Skalierung des Index in der Adressrechnung. (2 Punkte)

Musterlösung und Bewertung

a) Von oben nach unten: `bgeu`, `2`, `t0` (auch `a0` beziehungsweise `x10` zulässig), `0`, `t5`, `t2`. Je ein Punkt. Für die dritte Lücke in `add t4, _____, t3` wird `a0` beziehungsweise `x10` ohne Punktabzug akzeptiert: Es enthält während der gesamten Schleife unverändert die Arraybasis und wird erst bei der Rückgabe überschrieben. Die folgende Musterfunktion verwendet `t0`:

```
sum_first:
    addi t0, a0, 0
    addi t1, x0, 0
    addi t2, x0, 0
sum_loop:
    bgeu t1, a1, sum_end
    slli t3, t1, 2
    add  t4, t0, t3
    lw   t5, 0(t4)
    add  t2, t2, t5
    addi t1, t1, 1
    jal  x0, sum_loop
sum_end:
    addi a0, t2, 0
    jalr x0, 0(ra)
```

Der Test vor dem Load schützt den Nullfall und das Arrayende. Der Shift um zwei bildet den Byteoffset; der Load liest dessen Speicherwort, die Addition aktualisiert die Summe. Ein Stackframe ist nicht nötig: Es gibt keine Unteraufrufe und keine geänderten callee-saved Register.

b) `n=0`: Ergebnis 0, kein Array-Load. `n=3`: Ergebnis $-2 + 5 + 7 = 10$, drei Array-Loads. Je 0,5 Punkte für die vier Angaben.

c) $A_2 = \texttt{0x3000} + 4 \cdot 2 = \texttt{0x3008}$; Wert 7. Ein Punkt für die Adresse, 0,5 Punkte für den Wert. Ein Element belegt vier Byte und der Speicher ist byteadressiert; deshalb entspricht Index zwei einem Abstand von acht Byte. 0,5 Punkte für diese Begründung. Die Adresse ist nicht der Inhalt des Elements.

Korrekturhinweis: Ein Grenzfehler wird am fehlerhaften Prüfungsteil bewertet. Nachfolgende korrekte Rechen- und Adressierungsarbeit bleibt separat bewertbar; ein Folgefehler wird nicht mehrfach abgezogen.

Aufgabe 4: Funktionsaufruf und Stacktrace (8 Punkte)

Vor `PC=0x1000` gilt `a0=4`, `s0=99`, `sp=0x2000`. Der Stackbereich ist beschreibbar. Alle Zeilen sind echte Vier-Byte-Instruktionen; die angegebenen Adressen und Labels gelten für diesen Papierausschnitt.

```
0x1000: jal ra, F
0x1004: addi t0, a0, 0
F:
0x1100: addi sp, sp, -16
0x1104: sw ra, 12(sp)
0x1108: sw s0, 8(sp)
0x110C: addi s0, a0, 0
0x1110: jal ra, Double
0x1114: add a0, a0, s0
0x1118: lw s0, 8(sp)
0x111C: lw ra, 12(sp)
0x1120: addi sp, sp, 16
0x1124: jalr x0, 0(ra)
Double:
0x1200: slli a0, a0, 1
0x1204: jalr x0, 0(ra)
```

a) Tragen Sie den Zustand unmittelbar **nach** der jeweiligen Instruktion ein. `PC danach` bezeichnet die als Nächstes auszuführende Adresse. Geben Sie `a0` und `s0` dezimal, die übrigen Größen hexadezimal an. (5 Punkte)

gerade ausgeführt	PC danach	a0	s0	ra	sp
0x1110					
0x1124					

b) Geben Sie für beide Stores die effektive Adresse und das dort abgelegte 32-Bit-Wort in Hexadezimalschreibweise an. (2 Punkte)

c) Nur die Instruktion `lw ra, 12(sp)` wird durch eine wirkungslose Instruktion ersetzt; alle angegebenen Adressen bleiben unverändert. Welche Adresse wählt dann der erste Sprung bei `0x1124`? (1 Punkt)

Musterlösung und Bewertung

gerade ausgeführt	PC danach	a0	s0	ra	sp
0x1110	0x1200	4	4	0x1114	0x1FF0
0x1124	0x1004	12	99	0x1004	0x2000

a) Je 0,5 Punkte pro Tabellenzelle. Double verdoppelt vier zu acht; F addiert den erhaltenen Eingang vier, Ergebnis zwölf.

b) Store bei 0x1104: Adresse 0x1FFC, Wort 0x00001004. Store bei 0x1108: Adresse 0x1FF8, Wort 0x00000063. Je 0,5 Punkte pro Adresse beziehungsweise Wort.

c) Ziel 0x1114 statt 0x1004, weil die innere Linkadresse in ra bleibt. Ein Punkt. Der Helferaufruf überschreibt den äußeren Link; der Epilog muss ihn vor der Rückkehr wiederherstellen.

Aufgabe 5: Datenpfad und Steuerwerk (8 Punkte)

Verwenden Sie den Single-Cycle-Kursausschnitt für `add`, `sub`, `and`, `or`, `lw`, `sw` und `beq`. `ALUSrc` wählt 0 für die zweite Registerquelle oder 1 für das Immediate; `MemtoReg` wählt 0 für das ALU-Ergebnis oder 1 für Speicherlesedaten. Register- und Speicherschreiben haben getrennte Freigaben.

ImmGen liefert bei Branches den fertigen vorzeichenerweiterten Byteoffset; danach erfolgt kein weiterer Shift. Die ALU subtrahiert die Registerwerte zum Gleichheitstest.

a) Füllen Sie die Hauptsteuerzeilen mit 0, 1 oder X (Don't Care). Bei einer unerheblichen Auswahl sind auch begründete konkrete Werte zulässig. (6 Punkte)

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
`lw`						
`beq`						

b) Ein `beq` steht bei `PC=0x4000`. Beide Quellen enthalten 13; ImmGen liefert −12. Bestimmen Sie `Zero`, `PCSrc` und den nächsten PC. (2 Punkte)

Musterlösung und Bewertung

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
`lw`	1	1	1	1	0	0
`beq`	0	X	0	0	0	1

a) Je 0,5 Punkte pro Zelle. Bei beq sind MemtoReg = 0 und = 1 ebenfalls korrekt, weil RegWrite = 0 die Übernahme sperrt. Andere Freigaben sind hier keine Don't Cares.

b) $13 - 13 = 0$: `Zero=1` (0,5 Punkte). PCSrc $=$ Branch $\wedge$ Zero $= 1$ (0,5 Punkte). Neuer PC: `0x4000` $- 12 =$ `0x3FF4` (1 Punkt). Das Branchziel wird aus dem alten PC gebildet.

Aufgabe 6: Pipeline und Laufzeit (10 Punkte)

Fünf Stufen: IF (Befehl holen), ID (decodieren und Register lesen), EX (rechnen), MEM (Datenzugriff), WB (Rückschreiben / Abschluss). Anfangs leere Pipeline, höchstens ein neuer Befehl pro Takt, getrennte Befehls- und Datenzugriffe; jeder Load braucht eine MEM-Stufe. Forwarding aus EX/MEM und MEM/WB und Load-Use-Erkennung sind vorhanden. Der jüngste passende Produzent ist maßgeblich; ist sein Wert noch nicht bereit, muss gewartet werden. Ein Schreibversuch auf `x0` erzeugt keine Forwardingquelle. Ein Load liefert seine Daten erst nach MEM. Der Stall hält PC und IF/ID und erzeugt eine Bubble in ID/EX; ältere Arbeit läuft weiter.

Vor dem Ausschnitt steht `s0=0x3000`; dort liegt das Wort 7:

```
lw   t0, 0(s0)
addi t0, t0, 5
add  a0, t0, t0
```

a) Tragen Sie die Stufenfolge ein und kennzeichnen Sie den Stall. Wiederholtes ID beziehungsweise IF mit Stall-Markierung ist zulässig. (5 Punkte)

Befehl / Takt	1	2	3	4	5	6	7	8
`lw t0, 0(s0)`								
`addi t0, t0, 5`								
`add a0, t0, t0`								

b) Begründen Sie den Stall und erklären Sie, warum zwischen `addi` und `add` kein weiterer Stall nötig ist. (2 Punkte)

c) Ein **anderer** anfangs leerer Pipeline-Lauf umfasst 100 gültig abgeschlossene Maschineninstruktionen, acht Load-Use-Stallzyklen und drei genommene Branches. Die Branch-Auflösung erfolgt in EX; jeder genommene Branch kostet zwei zusätzliche Takte. Die Zusatzkosten überschneiden sich nicht, weitere Stalls gibt es nicht. Die Taktperiode beträgt 2 ns. Bestimmen Sie Gesamttakte, Gesamtzeit und Speedup gegenüber einer Ausführung derselben 100 Instruktionen mit CPI eins und 8 ns Taktperiode. Gezählt wird vom ersten IF bis Ende des letzten WB; Bubbles und verworfene Einträge zählen nicht zu den 100 Instruktionen. (3 Punkte)

Musterlösung und Bewertung

Befehl / Takt	1	2	3	4	5	6	7	8
`lw`	IF	ID	EX	MEM	WB			
`addi`		IF	ID	ID*	EX	MEM	WB	
`add`			IF	IF*	ID	EX	MEM	WB

a) • bedeutet gehalten; in Takt 4 liegt eine wirkungslose Bubble in EX. Ein Punkt für den vollständigen Load-Durchlauf, zwei für den addi-Durchlauf mit passendem Stall (je ein Punkt für Stufenlage und Halt), zwei für den entsprechend verschobenen add-Durchlauf (je ein Punkt für Stufenlage und Halt). „STALL“ anstelle von ID* beziehungsweise IF* ist gleichwertig.

b) Das Loadwort ist vor Ende von Takt 4 noch nicht verfügbar und erreicht addi in EX bei Takt 5 aus MEM/WB. Ein Punkt. Das ALU-Ergebnis von addi liegt bereits nach dessen EX vor und wird in Takt 6 aus EX/MEM an beide benutzten Quellen von add weitergereicht. Ein Punkt.

In Takt 5 fallen zwar das WB von `lw` und das ID von `add` zusammen. Für `add` ist jedoch der jüngere Produzent `addi` maßgeblich. Dessen Ergebnis wird in Takt 6 weitergeleitet; der ältere Loadwert darf nicht verwendet werden. Das Raster gilt daher sowohl bei Registerlesen erst im Folgetakt nach WB als auch bei Schreiben vor Lesen innerhalb desselben Takts. Die Aufgabe prüft Load-Use und EX-Forwarding; sie erfordert keinen zusätzlichen Stall wegen einer WB/ID-Konvention.

c) $C = N + 4 + S + 2B = 100 + 4 + 8 + 2 \cdot 3 = 118$ Takte. $t = 118 \cdot 2\,\text{ns} = 236\,\text{ns}$. Vergleichszeit $100 \cdot 1 \cdot 8\,\text{ns} = 800\,\text{ns}$, Speedup $800/236 \approx 3{,}39$. Je ein Punkt für Taktzahl, Gesamtzeit und Speedup. Die übrigen Teilgrößen werden bei plausibler Weiterrechnung mit einem zuvor falschen C nicht erneut abgezogen.

Aufgabe 7: Store codieren und im Speicher darstellen (6 Punkte)

Ein Array aus sechs 32-Bit-Wörtern beginnt bei `0x6000`; die Indizes beginnen bei null. `s1` (`x9`) zeigt auf das Element mit Index fünf. `a2` (`x12`) enthält den Wert 13, der das Element mit Index zwei überschreiben soll. Alle Speicherzugriffe sind gültig und ausgerichtet. Verwenden Sie `s1` als Basisregister.

a) Bestimmen Sie die Zieladresse und den vorzeichenbehafteten Byteoffset relativ zu `s1`. (1 Punkt)

b) Geben Sie eine einzelne native RV32I-Instruktion für diesen Speicherzugriff an. Codieren Sie sie: Benennen Sie die Felder des verwendeten Formats, tragen Sie deren Inhalte binär ein und bilden Sie das vollständige 32-Bit-Maschinenwort in Hexadezimalschreibweise. Nutzen Sie das ausgeteilte Referenzblatt. (4 Punkte)

c) Das Instruktionswort aus b) liegt ab Byteadresse `0x1800` im Little-Endian-Speicher. Geben Sie die vier Bytewerte mit ihren Adressen an. (1 Punkt)

Musterlösung und Bewertung

a) `s1` enthält $0\text{x}6000 + 4 \cdot 5 = 0\text{x}6014$. Die Zieladresse ist $0\text{x}6000 + 4 \cdot 2 = 0\text{x}6008$ (0,5 Punkte). Der benötigte Byteoffset ist $0\text{x}6008 - 0\text{x}6014 = -12$ (0,5 Punkte). Er zählt Byte, nicht Arrayelemente.

b) `sw a2, -12(s1)`, gleichwertig `sw x12, -12(x9)` (1 Punkt). Der Wert 13 bestimmt nicht die Registerfelder; codiert werden die Nummern der Register, die Adresse beziehungsweise Schreibdaten bereitstellen.

Das zwölfbittige Immediate ist $4096 - 12 = 4084$, also `0xFF4` = `1111 1111 0100`. Im S-Format verteilen sich seine Bits auf zwei Felder:

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
1111111	01100	01001	010	10100	0100011

Je 0,5 Punkte für die beiden Immediate-Teile. Ein Punkt für Register- und Funktionsfelder mit richtiger Feldanordnung (je 0,5 für Quellenzuordnung und übrige Felder/Anordnung). `rs1` wählt die Basis `s1`, `rs2` die Schreibdaten `a2`; ein `rd`-Feld existiert bei `sw` nicht.

Das zusammengesetzte Maschinenwort ist `0xFEC4AA23` (1 Punkt).

c) `0x1800`: `0x23`, `0x1801`: `0xAA`, `0x1802`: `0xC4`, `0x1803`: `0xFE`. Je 0,25 Punkte. Gefragt sind die Bytes der Instruktion, nicht die Bytes des gespeicherten Datenwerts 13.

Korrekturhinweis: Ein falsch berechneter Offset wird in a) bewertet. Die korrekte Codierung des eigenen Offsets einschließlich Immediate-Aufteilung und Hexzusammensetzung bleibt separat bewertbar. Bei c) wird die Little-Endian-Zerlegung des in b) angegebenen Instruktionsworts bewertet; ein vorheriger Codierfehler wird nicht erneut abgezogen.

Themenabdeckung und Zeitrahmen

Diese Übersicht dient der Korrektur und enthält keine zusätzlichen Aufgaben.

Einheit	Inhalte in den Multiple-Choice-Fragen	Vertiefung auf Papier
1	1–3, 5: Abstraktion, Bitmuster, Logik, Speicherung, Byteordnung	Aufgabe 2
2	1, 4: Register, Immediates und Codierung	Aufgaben 3 und 7
3	5–6: Erweiterung, Speicher und Branches	Aufgaben 3, 5 und 7
4	7: ABI und Registererhaltung	Aufgaben 3–4
5	1: ISA und Umsetzung	Aufgabe 5
6	8: Hauptsteuerung und ALU-Steuerung	Aufgabe 5
7	9–10: Laufzeit, CPI und Forwarding-Priorität	Aufgabe 6

Geplante Verteilung: zehn Minuten für zehn Auswahlfragen; 50 Minuten für sechs Papieraufgaben mit 50 Punkten. Die Punkte folgen dem Arbeitsaufwand und der Zahl eigenständiger Schritte. Kleine Einzelangaben erhalten entsprechend halbe oder Viertelpunkte. Der Zeitrahmen ist eine didaktische Schätzung für vorbereitete Studierende, keine mit einer Studierendengruppe gemessene Bearbeitungszeit.

Bei Papieraufgaben werden äquivalente fachlich richtige Lösungen akzeptiert. Folgefehler werden nur am ersten fehlerhaften Schritt abgezogen, soweit die Weiterrechnung die übrigen gefragten Fähigkeiten zeigt. Der Zeitrahmen und die Hilfsmittel dieser Probeklausur legen keine abweichenden Regeln einer späteren regulären Prüfung fest.